Follant

Organisation & Member Management SaaS

Technical Architecture & Operations Manual

Version 1.0 — June 2026

Multi-tenant admin platform for schools, charities, and businesses.
Stack: React 18, TypeScript, Vite, Supabase (Postgres, Auth, Storage, RLS), Vercel.

Contents

1 Executive Summary **1**

2 System Architecture **2**
2.1 High-Level Topology . . . 2
2.2 Runtime Modes . . . 2
2.2.1 Supabase Mode (Production) . . . 2
2.2.2 Legacy File-DB Mode (Local Demo) . . . 2
2.3 Request Flow (Supabase Mode) . . . 3

3 Functional Requirements **4**
3.1 Authentication & Access . . . 4
3.2 Organisation Management . . . 4
3.3 Member Management . . . 4
3.4 Audit & Reporting . . . 5
3.5 Account Settings . . . 5

4 Application Structure **6**
4.1 Route Map . . . 6
4.2 Key Source Modules . . . 6
4.3 UI State Pattern . . . 6

5 Data Model **8**
5.1 Core Tables . . . 8
5.1.1 profiles . . . 8
5.1.2 organizations . . . 8
5.1.3 organization_members . . . 8
5.1.4 access_profiles . . . 8
5.1.5 activity_logs . . . 8
5.1.6 activity_action_catalog . . . 8
5.1.7 uploaded_files . . . 8
5.1.8 user_sessions . . . 8
5.2 Organisation Type Constraints . . . 9

6 Security Architecture **10**
6.1 Row Level Security (RLS) . . . 10
6.2 Privilege Escalation Guards . . . 10
6.3 Audit Integrity . . . 10
6.4 Storage Security . . . 11
6.5 Express Legacy Security . . . 11

7 Authentication Flows **12**
7.1 Platform Admin Sign-In . . . 12
7.2 Invite Acceptance . . . 12
7.3 Password Reset . . . 12

8 Theming & User Interface **13**
8.1 Dark Mode Architecture . . . 13
8.2 Design Tokens . . . 13

9 Database Migrations **14**
9.1 Applying Migrations . . . 14
9.2 Platform Admin Bootstrap . . . 14

10 Edge Functions **15**

11 File Uploads **16**
11.1 Supabase Storage Flow . . . 16

12 Activity Log Reference **17**
12.1 Action Catalog (Selected) . . . 17

13 Environment Configuration **18**
13.1 Client Variables (Vite) . . . 18
13.2 Server-Only Variables . . . 18
13.3 Environment Separation . . . 18

14 Deployment **19**
14.1 Vercel . . . 19
14.2 Supabase Auth URLs . . . 19
14.3 Pre-Production Checklist . . . 19

15 Development & Operations **20**
15.1 Local Development . . . 20
15.2 Useful Scripts . . . 20
15.3 Data Import . . . 20
15.4 Type Generation . . . 20

16 Implementation Changelog **21**
16.1 2026-06-07 — Dark Mode, Query States, Supabase Environments . . . 21
16.2 2026-06-07 (b) — Dark Mode Reliability . . . 21
16.3 2026-06-07 (c) — Monochrome Theme . . . 21
16.4 2026-06-08 — Supabase Production Hardening . . . 21

17 Known Limitations & Roadmap **23**

18 Appendix A — Access Profile Permissions **24**

19 Appendix B — NPM Dependencies (Production) **25**

20 Appendix C — Glossary **26**

List of Tables

Chapter 1

Executive Summary

Follant (also referenced in internal materials as *ImpactOp* or *AdminDB*) is a software-as-a-service platform that enables platform administrators to create organisations, invite members by email, manage organisation settings and media assets, and maintain a tamper-resistant audit trail of user actions.

The product targets three organisation types:

- **School** — requires local authority or trust name and optional academic metadata.
- **Charity (Nonprofit)** — requires charity registration number (EIN-style format).
- **Business** — requires Companies House or equivalent registration number.

Production deployment uses a static React single-page application on Vercel, backed entirely by Supabase for authentication, relational data, row-level security, object storage, and optional Edge Functions. A legacy Express + JSON file database mode remains available for local development when Supabase environment variables are not configured.

Chapter 2

System Architecture

2.1 High-Level Topology

Layer	Technology
Client	React 18, TypeScript (strict), Vite (SWC)
Routing	React Router v6
UI components	shadcn/ui, Tailwind CSS v4, Lucide icons
Client state	TanStack React Query v5
Forms & validation	React Hook Form + Zod v4
Theming	next-themes, CSS custom properties
Primary backend	Supabase (Postgres + Auth + Storage)
Optional server logic	Supabase Edge Functions (Deno)
Legacy dev backend	Express 4 + `db.json` (disabled when Supabase configured)
Deployment	Vercel (static `dist/` SPA)

2.2 Runtime Modes

The application detects its backend at runtime via `src/lib/env.ts` and `src/server/supabase-mode.ts`.

2.2.1 Supabase Mode (Production)

When `VITE_SUPABASE_URL_*` and `VITE_SUPABASE_ANON_KEY_*` are present:

- The browser uses `@supabase/supabase-js` with persisted sessions.
- All data operations go through PostgREST with Row Level Security enforced.
- Organisation creation and member invites use direct Supabase client calls and RPCs (Edge Functions optional).
- File uploads use the `Pics` Storage bucket.
- The Express `/api/*` routes are disabled (`supabaseOnly` flag in `server.ts`).

2.2.2 Legacy File-DB Mode (Local Demo)

When Supabase variables are absent:

- Express serves REST endpoints backed by `db.json`.
- Sessions use HMAC-signed tokens with scrypt password hashing.

- Uploads are stored on the local filesystem under `uploads/`.
- Seeded credentials: `admin@example.com` / `Password123!`

2.3 Request Flow (Supabase Mode)

1. User signs in via Supabase Auth (`signInWithPassword`).
2. `AuthContext` loads the `profiles` row and stores the JWT.
3. React Query hooks call `src/lib/supabase-data.ts` and `supabase-storage.ts`.
4. PostgREST applies RLS policies using `auth.uid()` from the JWT.
5. Mutations invalidate relevant query keys; activity is logged via `log_activity()` RPC.

Chapter 3

Functional Requirements

3.1 Authentication & Access

Requirement	Implementation
Sign in	Supabase Auth or legacy Express session
Invite-only registration	Public sign-up redirects to `/accept-invite`; accounts created via invitation link
Password reset	Supabase `resetPasswordForEmail` or legacy email token flow
Admin-only dashboard	`ProtectedLayout` requires `profiles.is_admin = true`
Account status gates	Deactivated/suspended users are signed out automatically
Session persistence	Supabase localStorage tokens; legacy `auth_token` key

3.2 Organisation Management

- Create organisations with type-specific validated fields (Zod + Postgres CHECK constraints).
- Directory lists organisations visible via RLS (owner, member, or platform admin).
- Organisation detail page: Members, Settings, Media, Activity tabs.
- Update organisation profile, contact fields, address, and status.
- Upload and delete logo and banner images (JPEG, PNG, WebP, GIF; max 5 MB).
- Permanent deletion with double verification (confirm dialog, type organisation name, final confirm).

3.3 Member Management

- Invite members by email with role (admin, member, viewer) and optional access profile.
- Optimistic UI updates during invite; duplicate invites blocked at DB level.
- Member statuses: invited, active, suspended, removed.

- Invite preview RPC (`get_invite_preview`) for accept-invite page.
- Secure acceptance via `accept_organization_invite` RPC (migration 009).
- Pending invites activated on sign-in via `activate_pending_invites`.

3.4 Audit & Reporting

- Central activity log with plain-language labels from `activity_action_catalog`.
- Severity levels: info, notice, warning, critical.
- Platform admins see all logs; org owners see org-scoped logs.
- Statistics dashboard aggregates organisation and member counts.

3.5 Account Settings

- Profile update (name, phone, job title, bio, timezone).
- Avatar upload/delete via Supabase Storage.
- Theme preference (light/dark) synced to `profiles.theme` and next-themes.
- Notification preferences, password change, session revocation, account deactivation.

Chapter 4

Application Structure

4.1 Route Map

Path	Access	Page
`/sign-in`	Public	SignInPage
`/accept-invite`	Public	AcceptInvitePage
`/forgot-password`	Public	ForgotPasswordPage
`/reset-password`	Public	ResetPasswordPage
`/orgs`	Protected	OrgDirectoryPage
`/orgs/new`	Protected	CreateOrgPage
`/orgs/:id`	Protected	OrgDetailPage
`/account`	Protected	AccountSettingsPage
`/activity`	Protected	ActivityLogPage
`/statistics`	Protected	StatisticsPage

4.2 Key Source Modules

Module	Responsibility
`src/contexts/AuthContext.tsx`	Dual-mode auth, profile loading, invite acceptance
`src/lib/supabase-data.ts`	CRUD for orgs, members, profiles, activity logs
`src/lib/supabase-storage.ts`	Image upload/delete, `uploaded_files` records
`src/lib/supabase-mappers.ts`	Snake_case DB rows to TypeScript domain types
`src/hooks/useOrganizations.ts`	React Query hooks for org operations
`src/components/QueryState.tsx`	Loading, error, empty state pattern
`src/components/ConfirmProvider.tsx`	Destructive action confirmation dialogs
`src/types.ts`	Zod schemas, enums, domain interfaces, activity catalog
`src/server/security.ts`	Scrypt passwords, HMAC sessions, rate limits, headers
`server.ts`	Express dev server, legacy API, Vite middleware

4.3 UI State Pattern

Pages backed by React Query use the `QueryState` wrapper:

```
const { data, isLoading, isError, error, refetch } = useMyQuery();
return (
  <QueryState isLoading={...} isError={...} data={data} onRetry={refetch}>
    {(items) => items.length === 0
      ? <EmptyState ... />
      : /* render */}
  </QueryState>
);
```

Semantic CSS classes (`.app-page`, `.app-card`, `.app-input`, `.app-muted`) ensure consistent light/dark theming without hardcoded Tailwind colour pairs on every element.

Chapter 5

Data Model

5.1 Core Tables

5.1.1 profiles

Linked 1:1 to `auth.users`. Stores display name, email, platform admin flag, avatar URL, preferences, account status, and access profile reference.

5.1.2 organizations

Stores organisation identity, type, status, contact and address fields, media URLs, type-specific columns, and `created_by` (FK to auth user).

5.1.3 organization_members

Join table between organisations and users (or pending emails). Unique constraint on `(organization_id, email)`. Tracks role, status, access profile, invitation metadata.

5.1.4 access_profiles

Role templates with JSONB `permissions`. System profiles: `platform_admin`, `org_owner`, `org_admin`, `org_member`, `org_viewer`.

5.1.5 activity_logs

Append-only audit trail. Enriched columns: `action_label`, `category`, `severity`, `actor_name`, `organization_name`. Inserts only via `log_activity()` security definer function.

5.1.6 activity_action_catalog

Maps technical action codes (e.g. `member.invite`) to human-readable labels for the UI.

5.1.7 uploaded_files

Metadata for Storage objects: entity type, MIME type, size, public URL, uploader.

5.1.8 user_sessions

Tracks active login sessions for revocation from account settings (legacy and Supabase modes).

5.2 Organisation Type Constraints

Postgres CHECK constraints enforce conditional required fields:

- **school**: `school_district` required when `type = school`.
- **nonprofit**: `nonprofit_ein` must match `^\d{2}-\d{7}$`.
- **business**: `business_reg_number` required when `type = business`.

Client-side Zod schemas in `createOrgSchema` and `updateOrgSchema` mirror these rules.

Chapter 6

Security Architecture

6.1 Row Level Security (RLS)

Every public table has RLS enabled. Migration `009_pentest_rls_hardening.sql` adds FORCE RLS and revokes broad `anon` grants.

Key helper functions:

- `is_platform_admin()` — checks `profiles.is_admin`.
- `can_view_organization(uuid)` — owner, active member, or platform admin.
- `can_manage_organization(uuid)` — owner or delegated admin permissions.
- `can_invite_members(uuid)` — permission check via `get_effective_permissions`.
- `get_effective_permissions(uuid)` — merges access profile JSON with member role.

6.2 Privilege Escalation Guards

Triggers protect sensitive columns:

- `guard_profile_sensitive_columns` — prevents non-admins from setting `is_admin`, `access_profile_id` or `account_status`.
- Migration `012_bootstrap_platform_admin.sql` allows service role and postgres to bootstrap the first platform admin.
- `handle_new_user` (post-009) sets `is_admin = false` for new signups by default.

6.3 Audit Integrity

Clients cannot INSERT directly into `activity_logs`. All entries go through `log_activity()`, which:

1. Verifies authentication.
2. Validates the action exists in `activity_action_catalog`.
3. Checks org access when `p_organization_id` is supplied.
4. Denormalises actor and organisation names at write time.

6.4 Storage Security

Bucket `Pics` policies (migration 007–008):

- Public read for avatar, logo, and banner URLs.
- Authenticated users may upload, update, and delete only their own objects.
- SVG uploads blocked (migration 008) to reduce XSS risk.

6.5 Express Legacy Security

When the legacy server runs:

- Passwords hashed with scrypt; legacy SHA-256 rehashed on login.
- HMAC-SHA256 session tokens with 24-hour TTL.
- Rate limiting on auth endpoints (30 requests per 15 minutes per IP).
- Security headers middleware; `x-powered-by` disabled.
- Production startup blocked without `SESSION_SECRET` (32+ characters).

Chapter 7

Authentication Flows

7.1 Platform Admin Sign-In

1. User submits email and password on `/sign-in`.
2. Supabase Auth returns JWT; `AuthContext` fetches `profiles` row.
3. If `is_admin` is false, `ProtectedLayout` blocks access with message.
4. `activate_pending_invites()` runs to link any pending org memberships.

7.2 Invite Acceptance

1. Admin sends invite; row inserted in `organization_members` with status `invited`.
2. Invitee opens `/accept-invite?org=<id>&email=<email>`.
3. `get_invite_preview` RPC validates pending invite.
4. New user signs up via Supabase Auth with org metadata.
5. `accept_organization_invite` RPC sets `user_id`, `status = active`, `joined_at`.

7.3 Password Reset

Supabase mode uses `resetPasswordForEmail` with redirect to `/reset-password`. Legacy mode uses local token storage in `src/server/email.ts`.

Chapter 8

Theming & User Interface

8.1 Dark Mode Architecture

Implemented June 2026 (see `documentation.md` changelog):

- `next-themes` toggles `class="dark"` on `<html>`.
- Tailwind v4 uses `@custom-variant dark (&:where(.dark, .dark *))`.
- CSS variables define monochrome palette: pure white/black backgrounds, neutral grays for muted text.
- Blocking script in `index.html` prevents flash of wrong theme.
- `ThemeSync` applies profile preference once per session unless user already chose via toggle.
- `ThemeToggle` in sidebar cycles light ↔ dark.

8.2 Design Tokens

Token	Light	Dark
`-app-bg`	`#ffffff`	`#000000`
`-app-fg`	`#0f172a`	`#ffffff`
`-app-card`	`#ffffff`	`#0a0a0a`
`-app-muted`	`#64748b`	`#a3a3a3`

Severity pills, type badges, and navigation use semantic classes (`.app-severity-*`, `.app-type-option*`) with no blue/indigo accents in dark mode.

Chapter 9

Database Migrations

Apply in order against a fresh or existing Supabase project:

#	File
001	`initial_schema.sql` — profiles, organisations, members, base RLS
002	`comprehensive_saas.sql` — activity logs, uploaded files, user sessions
003	`rbac_audit_access.sql` — access profiles, RBAC helpers, activity catalog
004	`access_control_finalize.sql` — RLS on catalog tables
005	`security_hardening.sql` — permission enumeration blocks
006	`directory_indexes.sql` — query performance indexes
007	`storage_pics_bucket.sql` — Storage bucket and policies
008	`storage_no_svg.sql` — block SVG uploads
009	`pentest_rls_hardening.sql` — invite RPCs, FORCE RLS, hardened triggers
010	`invite_reactivate_removed.sql` — re-invite removed members
011	`invite_preview_enhanced.sql` — richer invite preview RPC
012	`bootstrap_platform_admin.sql` — admin promotion bootstrap
013	`fix_uuid_defaults.sql` — pgcrypto, gen_random_uuid, org.delete catalog

9.1 Applying Migrations

Via Supabase CLI:

```
npx supabase login
npx supabase link --project-ref <your-project-ref>
npx supabase db push
```

Or paste each file into the Supabase SQL Editor. Verify applied versions:

```
select version from supabase_migrations.schema_migrations order by version;
```

9.2 Platform Admin Bootstrap

After migrations, promote your production admin once:

```
update public.profiles
set is_admin = true
where email = 'your-admin@example.com';
```

Migration 012 automates this for `admin@example.com` and fixes the profile guard trigger for service-role updates.

Chapter 10

Edge Functions

Optional Deno functions in `supabase/functions/`:

Function	Purpose
`create-organization`	Validated org insert with activity log
`invite-member`	Permission-checked member invite + email stub

Deploy:

```
npm run supabase:functions:deploy
```

When Edge Functions are not deployed, the React client performs equivalent operations via direct Supabase inserts and RPCs (`invokeCreateOrganization`, `invokeInviteMember` in `supabase-data.ts`).

Required Edge Function secrets:

```
APP_URL=https://your-app.vercel.app
ALLOWED_ORIGINS=http://localhost:3000,https://your-app.vercel.app
```

Chapter 11

File Uploads

11.1 Supabase Storage Flow

1. Client validates MIME type and 5 MB size limit.
2. File uploaded to `Pics` bucket at path `{userId}/orgs/{orgId}/{logo|banner}/{uuid}.{ext}`.
3. Row inserted into `uploaded_files` with explicit `id` (client-generated UUID).
4. Organisation row updated with public URL.
5. `log_activity` called with `org.logo_upload` or `org.banner_upload`.
6. Previous image removed from Storage and `uploaded_files` if replacing.

Migration 013 ensures hosted databases have `pgcrypto` and `gen_random_uuid()` for server-side UUID generation in `log_activity`.

Chapter 12

Activity Log Reference

12.1 Action Catalog (Selected)

Action	Label	Category	Severity
`auth.sign_in`	Signed in	Authentication	info
`org.create`	Organisation created	Organisations	notice
`org.delete`	Organisation deleted	Organisations	critical
`member.invite`	Member invited	Members	notice
`member.remove`	Member removed	Members	warning
`org.logo_upload`	Logo uploaded	Organisations	info
`file.delete`	File deleted	Files	warning
`account.deactivate`	Account deactivated	Security	critical

Full catalog defined in `src/types.ts` (`ACTIVITY_ACTION_CATALOG`) and seeded in migration 003.

Chapter 13

Environment Configuration

13.1 Client Variables (Vite)

Variable	Description
`VITE_SUPABASE_URL_DEV`	Supabase project URL (development builds)
`VITE_SUPABASE_ANON_KEY_DEV`	Public anon key (development)
`VITE_SUPABASE_URL_PROD`	Supabase project URL (production builds)
`VITE_SUPABASE_ANON_KEY_PROD`	Public anon key (production)
`VITE_SUPABASE_STORAGE_BUCKET_*`	Storage bucket name (default: `Pics`)
`VITE_SUPABASE_SCHEMA_*`	Optional schema override (single-project dual-env)

Resolution logic in `getSupabaseConfig()` uses `import.meta.env.MODE` to select `_DEV` or `_PROD` keys, with generic fallbacks.

13.2 Server-Only Variables

- `SUPABASE_SERVICE_ROLE_KEY_*` — CLI, import scripts, Edge Functions only. Never prefix with `VITE_`.
- `SESSION_SECRET` — Express legacy mode (32+ characters in production).
- `APP_URL` — invite and password-reset link base URL.
- `PORT` — Express dev server port (default 3000).

13.3 Environment Separation

Recommended: two separate Supabase projects (development and production). See `supabase/environments.md`.

Git branching alignment:

- `main` → production Vercel URL → prod Supabase project.
- `development` → preview URL → dev Supabase project.
- `feature/*` → short-lived branches merged via PR.

Chapter 14

Deployment

14.1 Vercel

`vercel.json` configuration:

- Build command: `npm run build:web` (`vite build` only).
- Output directory: `dist/`.
- SPA rewrites send all non-asset routes to `index.html`.

Production environment variables must be set in Vercel before build. Vite embeds `VITE_*` at compile time; redeploy after any change.

14.2 Supabase Auth URLs

In Supabase Dashboard → Authentication → URL Configuration:

- Site URL: `https://your-app.vercel.app`
- Redirect URLs: `http://localhost:3000/**`, `https://your-app.vercel.app/**`, preview URLs as needed.

14.3 Pre-Production Checklist

1. Apply migrations 001–013 on the production Supabase project.
2. Promote platform admin (`is_admin = true`).
3. Set Vercel `VITE_SUPABASE_*` production variables.
4. Configure Supabase Auth redirect URLs for production domain.
5. Verify Storage bucket `Pics` exists with correct policies.
6. (Optional) Deploy Edge Functions and set `APP_URL`, `ALLOWED_ORIGINS` secrets.
7. Smoke test: sign-in, org CRUD, invite, image upload, activity log, org delete.
8. Replace dev-only credentials; do not use `admin@example.com` in production.

Chapter 15

Development & Operations

15.1 Local Development

```
npm install
cp .env.example .env.development
# Fill VITE_SUPABASE_URL_DEV and VITE_SUPABASE_ANON_KEY_DEV
npm run dev
```

Without Supabase vars, the app runs Express + `db.json` on port 3000 with Vite HMR.

15.2 Useful Scripts

Command	Purpose
`npm run lint`	TypeScript check (`tsc -noEmit`)
`npm run build`	Vite build + Express bundle
`npm run build:web`	Vite build only (Vercel)
`npm run import:db-to-supabase`	One-time `db.json` → Supabase import
`npm run supabase:db:push`	Apply pending migrations
`npm run supabase:functions:deploy`	Deploy Edge Functions

15.3 Data Import

`scripts/import-db-to-supabase.mjs` imports organisations, members, and activity logs from local `db.json` using the service role key. Remaps legacy admin user ID to the Supabase Auth user ID.

15.4 Type Generation

After schema changes:

```
npx supabase gen types typescript --project-id <ref> > src/lib/database.types.ts
```

Chapter 16

Implementation Changelog

This chapter summarises incremental changes recorded in `documentation.md`.

16.1 2026-06-07 — Dark Mode, Query States, Supabase Environments

Dark mode: Added `ThemeProvider`, `ThemeSync`, `ThemeToggle`. Semantic CSS classes in `index.css`. User theme stored in `profiles.theme`.

Query states: Introduced `QueryState.tsx` with `LoadingState`, `ErrorState`, `EmptyState`. Applied to directory, detail, activity, and protected layout pages.

Supabase environments: Added `src/lib/env.ts`, `src/lib/supabase.ts`, environment example files, and `supabase/environments.md`. Dual-mode runtime preserves legacy file-DB when Supabase is not configured.

16.2 2026-06-07 (b) — Dark Mode Reliability

Fixed theme toggle being overwritten by `ThemeSync`. Switched to CSS variable-based monochrome palette. Added blocking script in `index.html` to prevent flash. Updated all major pages to semantic surface classes.

16.3 2026-06-07 (c) — Monochrome Theme

Removed blue/indigo accents from dark mode. Expanded semantic utilities for tabs, severity pills, type selectors, and form controls. Verified across sign-in, directory, detail, create, account, and activity pages.

16.4 2026-06-08 — Supabase Production Hardening

- Full Supabase client data layer (`supabase-data.ts`, `supabase-storage.ts`).
- Auth loading fix: `getSession()` with guaranteed `setSupabaseLoading(false)`.
- Direct org create and invite (no Edge Function dependency).
- Activity logging wired to org views, creates, invites, uploads.
- Organisation delete with double verification in Settings danger zone.
- Migration 013: UUID defaults, `log_activity` fix, `org.delete` catalog entry.

- Client-side UUID on `uploaded_files` insert for hosted DB compatibility.

Chapter 17

Known Limitations & Roadmap

- **Invite emails:** Edge Function email delivery is stubbed; invites appear in UI with `emailSent: false` unless a provider is integrated.
- **Edge Functions:** Optional; client uses direct Supabase calls as fallback.
- **E2E tests:** Not yet implemented (Playwright recommended for theme and query-state flows).
- **Bundle size:** Main JS chunk exceeds 500 KB; logo asset is large (~950 KB PNG).
- **database.types.ts:** Manual regeneration recommended after each migration batch.
- **Separate dev/prod Supabase projects:** Recommended but not enforced by tooling.

Chapter 18

Appendix A — Access Profile Permissions

Permissions are stored as JSONB. Example structure for `org_admin`:

```
{
  "organizations": { "read": true, "update": true, "delete": true },
  "members": { "invite": true, "update": true, "remove": true },
  "activity_logs": { "read": true },
  "files": { "upload": true, "delete": true },
  "settings": { "read": true, "write": true }
}
```

`get_effective_permissions(organization_id)` merges the member's access profile with org ownership rules and is used by RLS policies and the invite flow.

Chapter 19

Appendix B — NPM Dependencies (Production)

Package	Version (approx.)
react / react-dom	18.3
react-router-dom	6.30
@supabase/supabase-js	2.107
@tanstack/react-query	5.101
react-hook-form	7.77
zod	4.4
next-themes	0.4
tailwindcss	4.1
lucide-react	0.546
express (dev server)	4.21

Chapter 20

Appendix C — Glossary

RLS
Row Level Security — Postgres feature enforcing per-row access in the database.

RPC
Remote Procedure Call — Supabase Postgres function invoked via PostgREST.

JWT
JSON Web Token — Bearer token from Supabase Auth identifying the user.

SPA
Single Page Application — client-rendered app with client-side routing.

RBAC
Role-Based Access Control — permissions via roles and access profiles.

Edge Function
Serverless Deno function hosted by Supabase at `/functions/v1/`.

Platform Admin
User with `profiles.is_admin = true`; full audit visibility.

Org Owner
User who created the organisation (`organizations.created_by`).